

Pre-Lab: Installing pavo and Friends

Complete BEFORE Wednesday May 6 — Lab 5: Pavo Introduction

EEB 187 — Ecology & Evolution of Color

Table of contents

Why this matters	1
What you need to install	2
Step 1: Install R (if you don't have it)	2
Step 2: Install RStudio	2
Step 3: Install the R packages	3
Step 4: Smoke test	3
Step 5 (optional but recommended): Create a lab project folder	4
Step 6 (RECOMMENDED): Install the segmentation tools	4
macOS	4
Windows	5
Linux	6
Recommended alternative: install uv instead of pip-installing rembg	6
If install fails entirely	7
Step 7 (REQUIRED): Identify three fish from your assigned lineage	7
Common errors and fixes	8
What we'll do in lab	8
Need help?	9

Why this matters

Wednesday's lab is a hands-on R session. **You must arrive with a working R + RStudio + pavo installation.** If you try to install during class, you will fall behind and miss the live walkthrough. Allocate about **30 minutes** the day before lab to do this. Most of that time is downloads.

If you hit problems, post in the EEB 187 channel — there's almost certainly a classmate fighting the same error.

What you need to install

Tool	Version	Why
R	4.3	The language pavo runs on
RStudio	Recent (2024+)	The editor we'll use
pavo	2.9	Color pattern analysis
tidyverse	Recent	Data wrangling + ggplot
magick	Recent	Image I/O backend pavo uses

Step 1: Install R (if you don't have it)

- **macOS:** <https://cran.r-project.org/bin/macosx/> — get the `.pkg` for your chip (Apple Silicon = “arm64”, Intel = “x86_64”)
- **Windows:** <https://cran.r-project.org/bin/windows/base/> — get the `.exe`
- **Linux:** use your distro's package manager (`apt install r-base`, etc.)

Verify by opening a terminal and running:

```
R --version
```

You should see something like `R version 4.3.x` or higher.

Step 2: Install RStudio

<https://posit.co/download/rstudio-desktop/> — pick the free desktop version. Install. Open it once to confirm it launches.

Step 3: Install the R packages

Open RStudio. In the Console pane (bottom-left), paste and run:

```
install.packages(c("pavo", "tidyverse", "magick", "knitr"))
```

This will take **5–15 minutes** depending on your connection. You'll see a lot of compilation output. That's normal. Wait for the prompt (`>`) to come back.

Warning

On macOS: if you see errors about `Cellar`, `gfortran`, or `clang`, you need Apple's command-line tools first. Run in Terminal: `xcode-select --install`, then re-run the R install.

On Windows: if `pavo` fails to install, you may need to install Rtools first: <https://cran.r-project.org/bin/windows/Rtools/>

Step 4: Smoke test

In the RStudio Console, paste and run **all** of this:

```
library(pavo)
library(magick)
library(tidyverse)

# Load a test image bundled with pavo
data(sicalis)

# Run a quick check of the pavo version
packageVersion("pavo")

cat("\nIf you see a version number above ( 2.9.0) and no errors,\n")
cat("you are ready for Wednesday's lab.\n")
```

You should see:

```
[1] '2.9.0' # or higher
```

If you see a version number above (2.9.0) and no errors, you are ready for Wednesday's lab.

If you got that, **stop**. You're done.

Step 5 (optional but recommended): Create a lab project folder

Pick a place on your computer where you keep coursework — Documents, Desktop, wherever. Make a folder called EEB187-pavo-lab. On Wednesday I'll have you download a ZIP from BruinLearn that you'll unpack into this folder. Doing this in advance saves 90 seconds of fumbling on the day.

```
Documents/  
  EEB187-pavo-lab/      ← create this empty folder now
```

Step 6 (RECOMMENDED): Install the segmentation tools

Your fish images need their backgrounds removed before pavo can analyze them cleanly. The lab uses two command-line tools, both invoked from R via a one-line helper. **Install both before lab if you can** — installing during lab will eat your work time.

macOS

i Don't have Homebrew yet? Most students won't.

Homebrew (**brew**) is the macOS package manager — a one-time install that makes installing things like ImageMagick a one-liner forever after. **You only need to do this once on your laptop.**

Install Homebrew (5 minutes):

1. Open **Terminal** (Applications → Utilities → Terminal, or hit -Space and type "Terminal").
2. Paste this command and press return:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/ins
```

3. It will ask for your **macOS password** (the one you use to log into your Mac). Type it — characters won't show as you type, that's normal — and press return.
4. The installer downloads and configures Homebrew. Watch for a message at the very end labeled “**Next steps**”. On Apple-Silicon Macs (M1/M2/M3/M4) it will tell you to run two more commands that look like this:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Run those exact commands (copy from your Terminal output, not from this guide — your username will be different). They put **brew** on your PATH so future Terminal sessions can find it.

5. Confirm it works:

```
brew --version          # should print "Homebrew 4.x.x"
```

If **brew --version** says “command not found,” your shell hasn't been told about Homebrew yet. **Quit Terminal and reopen it**, then try again. If still no luck, go back and re-do step 4 (the **eval** line).

Full official instructions, in case anything looks different on your machine: <https://brew.sh>

Once Homebrew is in place:

```
# In Terminal:
brew install imagemagick
pip install "rembg[cpu,cli]"          # or pipx install "rembg[cpu,cli]"
```

Verify both work:

```
magick -version          # should print "Version: ImageMagick 7.x..."
rembg --help             # should print rembg's usage text
```

Windows

Install [ImageMagick from the official site](#) (the “Windows Binary Release” installer), then in Command Prompt:

```
pip install "rembg[cpu,cli]"
```

Linux

```
sudo apt install imagemagick  
pip install "rembg[cpu,cli]"
```

Recommended alternative: install uv instead of pip-installing rembg

Several students will have trouble with `pip install "rembg[cpu,cli]"` because `rembg`'s deep-learning dependencies (`onnxruntime`, `opencv`) don't always have prebuilt wheels for the latest Python. There's a much smoother path that **handles all of this automatically**:

```
# macOS:  
brew install uv  
  
# Linux:  
curl -LsSf https://astral.sh/uv/install.sh | sh  
  
# Windows:  
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

That's it. With `uv` installed, the `segment_fish()` helper in lab will run `rembg` via `uvx --from "rembg[cpu,cli]" rembg ...` — `uv` downloads a compatible Python and `rembg` automatically the first time you call the helper, and caches both for future calls. **No more pip install errors, no Python version mismatches.**

`segment_fish()` tries `uvx` first and falls back to a direct `rembg` binary, so if you got `rembg` installed via `pip` and it's on `PATH`, that works too.

Common roadblock 1 — Windows + pip install fails on rembg

If you tried `pip install "rembg[cpu,cli]"` on Windows and saw errors about Visual C++ build tools, ONNX runtime, or `opencv`: **don't fight it**. Install `uv` instead (one PowerShell command above) and skip `pip` entirely. `segment_fish()` will use `uvx` automatically.

 Common roadblock 2 — “rembg installed but R can’t find it”

If `rembg --help` works in your terminal but R says `rembg failed` (or `not installed`): R doesn’t see your full PATH. Check what R sees:

```
Sys.getenv("PATH")
```

If `/opt/homebrew/bin` (macOS Homebrew) or `~/local/bin` (pip user installs) isn’t in there, **install uv and use that path instead** — `uvx` is more discoverable. Or set the PATH in `~/.Renviron`:

```
PATH=/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:~/local/bin
```

Restart RStudio for the change to take effect.

 Common roadblock 3 — “I had to fall back to manual cropping; how does that affect my answers?”

If neither `uv` nor `rembg` work for you, the worksheet’s **manual-crop path** (Step 1.5 Path 2) is fully supported. Your A values will be less dramatic than the demo (closer to 1 across the board) because some background pixels will pollute the analysis. **The directional A claim still holds** — vertical-bar fish > 1 , horizontal-stripe fish < 1 — just with smaller effect sizes. Note in your worksheet which path you used; the rubric accommodates both.

If install fails entirely

Come to lab anyway with your three lineage fish images saved locally. Use the manual-crop path. The lab is still teachable; just plan for the smaller-magnitude metrics described above.

Step 7 (REQUIRED): Identify three fish from your assigned lineage

Your group has been assigned a reef fish lineage (typically a family). Before lab on Wednesday, **pick three species from your lineage that show three visually different pattern types** — for example:

- One with strong **vertical bars** (you’ll predict $A > 1$)
- One with a **horizontal stripe or band** (you’ll predict $A < 1$)
- One that’s **spotted, reticulate, or solid** (you’ll predict $A \approx 1$)

If your lineage doesn't span all three pattern types, pick the three with the most different patterns you can find in it.

Image source — use whichever your group has been working with:

- **FishView server** — if your group has been curating images there, use that
- **FishBase** ([fishbase.se](https://www.fishbase.org/)) or **Wikimedia Commons** — search by species name; download a clear, lateral-view photo
- **Fallback:** the lab bundle includes pre-curated images at `images/student-fish/<your-lineage>/` — five species per lineage, ready to use

Save your three images locally before lab — either in your `EEB187-pavo-lab` folder or somewhere you can find them quickly. **Don't show up empty-handed.** If you don't know your lineage assignment, ask in the channel by Tuesday evening.

Common errors and fixes

there is no package called 'pavo' — `install.packages("pavo")` didn't actually run. Re-run it; check for a typo.

unable to load shared object ... magick.so — On macOS, install ImageMagick via Homebrew first: `brew install imagemagick`, then reinstall the R `magick` package. On Linux: `sudo apt install libmagick++-dev`.

installation of package 'XXX' had non-zero exit status — The package failed to compile. Scroll up in the install output to see the actual error. Most common: missing system dependency. Google the error or post in the channel.

RStudio crashes on startup — Reinstall RStudio. If still failing, your R install is broken; reinstall R, then RStudio.

What we'll do in lab

You don't need to read up on pavo or do prep beyond Steps 1–7 above. Wednesday's lab walks you through everything from scratch:

1. **Demo block** — three reef fish from three families: sergeant major (vertical bars), cleaner wrasse (horizontal stripe), mandarin fish (reticulate). All three are pre-segmented for you.

2. **Predict** which pattern statistic will tell them apart (spoiler: the **A** statistic)
 3. Run pavo's `classify()` and `adjacent()` on all three demos; compare predictions to measurements
 4. **Your turn** — segment your three lineage fish using `segment_fish()` (or manual crop), then run the same pipeline
 5. Compare your three fish's metrics to the demo fish — does the A statistic correctly classify your fish by pattern orientation, the way it did for the demo?
 6. Submit a worksheet with three metric tables, a comparison answer, and a biological interpretation
-

Need help?

Post in the EEB 187 channel before Wednesday. Don't show up Wednesday with a broken install — you'll spend the lab on installation instead of biology.